Wydział Matematyki i Nauk Informacyjnych
POLITECHNIKA WARSZAWSKA

KONSPEKT PROJEKTU BADAWCZEGO

# Zastosowanie algorytmu mrówkowego do rozwiązania problemu *Capacitated Vehicle Routing Problem* *(CVRP)*

**Hubert Sobociński**
*Numer indeksu:* 327315

Warszawa, 15 marca 2026

# Spis treści

**1 Opis problemu** . . . . . **2**

**2 Wstępny przegląd literatury** . . . . . **3**
2.1 Metody dokładne . . . . . 3
2.2 Algorytmy heurystyczne . . . . . 3
2.3 Metaheurystyki dla CVRP . . . . . 3

**3 Algorytm mrówkowy – opis działania** . . . . . **4**
3.1 Intuicja biologiczna . . . . . 4
3.2 Ogólny schemat algorytmu ACO dla CVRP . . . . . 4
3.3 Prawdopodobieństwo wyboru następnego węzła . . . . . 5
3.4 Zbiór dozwolonych węzłów $\mathcal{N}_i^k$ . . . . . 5
3.5 Wariant 1 – podstawowy Ant System (AS) . . . . . 6
3.6 Wariant 2 – MAX-MIN Ant System (MMAS) . . . . . 7
3.7 Wariant 3 – ACO z inicjalizacją przez klasteryzację (K-Means+ACO) 7
3.8 Parametry algorytmu i ich wpływ . . . . . 8

**4 Hipotezy badawcze** . . . . . **8**

**5 Sposób weryfikacji hipotez** . . . . . **9**
5.1 Zbiory testowe i ocena jakości . . . . . 9
5.2 Porównywane algorytmy . . . . . 9
5.3 Protokół eksperymentów . . . . . 10

**6 Harmonogram projektu** . . . . . **10**

**7 Ogólny projekt techniczny** . . . . . **10**
7.1 Język i środowisko . . . . . 10
7.2 Planowane biblioteki . . . . . 11
7.3 Struktura kodu . . . . . 11
7.4 Reprodukowalność . . . . . 11

# 1. Opis problemu

**Capacitated Vehicle Routing Problem (CVRP)** jest szczególnym przypadkiem problemu trasowania pojazdów (*Vehicle Routing Problem*, VRP), w którym dodatkowo uwzględnia się ograniczenie pojemności pojazdów. Problem polega na wyznaczeniu optymalnych tras dla floty pojazdów obsługujących zbiór klientów rozmieszczonych w przestrzeni dwuwymiarowej oraz jeden centralny magazyn (depot).

Dysponujemy flotą $K$ pojazdów, z których każdy rozpoczyna trasę w magazynie i po jej zakończeniu musi do niego powrócić. Każdy pojazd może pokonać trasę o łącznej długości nieprzekraczającej $S_{\max}$ (ograniczenie dystansu). Każdy klient $i$ charakteryzuje się zapotrzebowaniem $d_i > 0$, natomiast każdy pojazd posiada maksymalną pojemność $C$.

Celem problemu jest przypisanie klientów do pojazdów oraz ustalenie kolejności ich obsługi w taki sposób, aby spełnione były następujące warunki:

1. każdy klient został obsłużony dokładnie jeden raz,
2. suma zapotrzebowań klientów obsługiwanych w ramach jednej trasy nie przekraczała pojemności pojazdu $C$,
3. całkowita długość każdej trasy nie przekraczała wartości $S_{\max}$,
4. łączna długość wszystkich tras była minimalna.

Formalnie problem można przedstawić w postaci grafu $G = (V, E)$, gdzie:

- $V = \{0, 1, 2, \ldots, n\}$ oznacza zbiór wierzchołków, przy czym 0 reprezentuje magazyn, a pozostałe wierzchołki odpowiadają klientom,
- $E$ oznacza zbiór krawędzi reprezentujących możliwe połączenia pomiędzy wierzchołkami,
- każdej krawędzi $(i, j)$ przypisana jest odległość $s_{ij}$,
- każdy klient $i$ posiada zapotrzebowanie $d_i$,
- każdy pojazd posiada pojemność $C$.

Jak wskazano w pracy [2], problem CVRP jest problemem NP-trudnym. Oznacza to, że wraz ze wzrostem liczby klientów czas potrzebny na znalezienie rozwiązania optymalnego rośnie wykładniczo. Z tego względu w praktyce najczęściej stosuje się algorytmy heurystyczne oraz metaheurystyczne, które pozwalają znaleźć dobre rozwiązania w rozsądnym czasie obliczeń. W niniejszej pracy do rozwiązania problemu CVRP wykorzystano algorytm mrówkowy (*Ant Colony Optimization*, ACO).

Problem CVRP znajduje szerokie zastosowanie w praktyce, szczególnie w logistyce oraz transporcie, gdzie wykorzystywany jest do optymalizacji planowania tras pojazdów i efektywnego zarządzania systemami dystrybucji.

# 2. Wstępny przegląd literatury

## 2.1. Metody dokładne

Metody dokładne gwarantują wynik optymalny, lecz ich złożoność obliczeniowa sprawia, że są praktyczne jedynie dla małych instancji ($n \lesssim 50$–$200$). Do najważniejszych należą:

- **Branch-and-bound / branch-and-cut** – algorytmy drzewa przeszukiwań wzmocnione cięciami płaszczyzn [4]. W połączeniu z zaawansowanymi separatorami nierówności pojemnościowych pozwalają rozwiązywać instancje do $n \approx 200$ [6].
- **Programowanie dynamiczne** – algorytm Helda–Karpa [5] jest dokładny dla TSP, jednak w CVRP z wieloma pojazdami wymaga przestrzeni stanów rzędu $2^n \cdot n$, co wyklucza stosowanie go już dla $n > 30$.
- **Column generation** – sformułowanie set-partitioning w połączeniu z branch-and-price umożliwia optymalne rozwiązanie instancji do kilkuset klientów [7].

Dla grafów o $n > 100$ konieczne staje się zatem stosowanie heurystyk i metaheurystyk.

## 2.2. Algorytmy heurystyczne

Heurystyki konstruktywne budują rozwiązanie w jednym przebiegu, bez powrotu do wcześniejszych decyzji. Najważniejsze z nich to:

- **Nearest Neighbor (NN)** [19] – każdy pojazd iteracyjnie odwiedza najbliższego nieobsłużonego klienta mieszczącego się w ramach ograniczeń pojemności i dystansu. Metoda jest prosta w implementacji, lecz jakość wyników jest typowo odległa od optimum o 20–30%.
- **Clarke–Wright Savings** [8] – algorytm oparty na pojęciu *oszczędności* wynikającej z połączenia dwóch tras w jedną. Generuje listę wszystkich możliwych połączeń sortowaną malejąco według wartości $s_{ij} = d_{0i} + d_{0j} - d_{ij}$, a następnie zachłannie scala trasy, o ile nie narusza to ograniczeń. Wyniki są przeciętnie lepsze niż NN.

## 2.3. Metaheurystyki dla CVRP

Metaheurystyki stanowią obecnie dominującą klasę metod dla CVRP, oferując dobre kompromisy między jakością rozwiązania a czasem obliczeń.

- **Przeszukiwanie lokalne i metaheurystyki trajektoryjne** – są to metody ulepszania istniejącego rozwiązania poprzez małe zmiany w trasie.

  Algorytm **2-opt** [9] usuwa skrzyżowania w trasie poprzez zamianę dwóch krawędzi. Bardziej zaawansowane wersje, takie jak **3-opt** oraz algorytm **Lin–Kernighan (LK)** [10], wykonują bardziej złożone zmiany w trasie, dzięki czemu często znajdują lepsze rozwiązania.

  Metoda **Tabu Search (TS)** [11] zapamiętuje ostatnio wykonane ruchy i tymczasowo ich zabrania, co pomaga uniknąć powrotu do tych samych rozwiązań

i wyjść z optimum lokalnego. Wariant *granular tabu* [12] dodatkowo ogranicza liczbę sprawdzanych połączeń, dzięki czemu algorytm działa szybciej i osiąga wyniki zbliżone do najlepszych znanych rozwiązań.

- **Algorytmy ewolucyjne i genetyczne** – są inspirowane procesem ewolucji. W algorytmach genetycznych rozwiązania są traktowane jak osobniki w populacji. Algorytm łączy je i modyfikuje (tzw. krzyżowanie i mutacje), aby tworzyć coraz lepsze trasy [13].

  Jednym z najlepszych obecnie algorytmów dla problemu VRP jest **HGS** (Hybrid Genetic Search) zaproponowany przez Vidala [14]. Algorytm ten uzyskał wiele najlepszych znanych wyników dla instancji z biblioteki CVRPLib.

- **Algorytmy mrówkowe (ACO)** – zostały zaproponowane przez Dorigo *i in.* [15]. Należą do metod inspirowanych zachowaniem mrówek, które podczas poszukiwania pożywienia zostawiają ślad feromonowy wskazujący dobrą drogę. Jednym z pierwszych ważnych wariantów był **Ant Colony System (ACS)** [16]. Później zaproponowano **MAX-MIN Ant System (MMAS)** [17], który ogranicza wartości feromonów, co pomaga uniknąć sytuacji, w której algorytm zbyt szybko skupia się tylko na jednej trasie. Dokładny opis działania ACO znajduje się w sekcji 3.

## 3. Algorytm mrówkowy – opis działania

### 3.1. Intuicja biologiczna

Algorytm mrówkowy (Ant Colony Optimization, ACO) zainspirowany jest zachowaniem kolonii mrówek podczas poszukiwania pożywienia [15]. Mrówki komunikują się pośrednio przez substancję chemiczną – **feromon**. Mechanizm działa następująco:

1. Mrówki wyruszają z gniazda w różnych kierunkach i poruszają się początkowo losowo.

2. Każda mrówka zostawia za sobą ślad feromonowy proporcjonalny do jakości znalezionej drogi – im krótsza trasa, tym więcej feromonów na jednostkę długości.

3. Kolejne mrówki preferują krawędzie z silniejszym śladem, co wzmacnia dobre ścieżki.

4. Feromon z czasem *paruje* – złe ścieżki są stopniowo zapominane, co zapobiega utknięciu w jednym rozwiązaniu.

W wyniku tych prostych reguł cała kolonia samoistnie koncentruje się na krótkich trasach bez żadnego centralnego planowania.

### 3.2. Ogólny schemat algorytmu ACO dla CVRP

W zastosowaniu do CVRP każda *mrówka* buduje kompletny zbiór tras obsługujący wszystkich klientów. Każda iteracja składa się z trzech kroków:

1. **Konstruowanie rozwiązań** – $m$ mrówek buduje trasy, wybierając kolejnych klientów probabilistycznie na podstawie feromonów i odległości. Każda mrówka szanuje ograniczenia pojemności $C$ i dystansu $S_{\max}$, gdy dodanie kolejnego klienta naruszyłoby któreś z nich, mrówka wraca do magazynu i rozpoczyna nowy pojazd.

2. **Ocena rozwiązań** – obliczana jest łączna długość tras każdej mrówki.

3. **Aktualizacja feromonów** – feromon na wszystkich krawędziach paruje, a następnie krawędzie wchodzące w skład dobrych tras są wzmacniane.

Powyższy cykl powtarzany jest przez $T_{\max}$ iteracji, a na końcu zwracane jest najlepsze znalezione rozwiązanie.

### 3.3. Prawdopodobieństwo wyboru następnego węzła

Podczas konstruowania trasy mrówka $k$ przebywająca w węźle $i$ wybiera następnego klienta $j$ losowo, ale z prawdopodobieństwem zależnym od dwóch czynników:

- **Feromon** $\tau_{ij}$ – im więcej feromonów na krawędzi $(i, j)$, tym ta krawędź wcześniej okazywała się lepsza.
- **Odległość** $\eta_{ij} = 1/d_{ij}$ – odwrotność odległości; zachęca mrówkę do wyboru bliższych klientów.

Stąd:

$$p_{ij}^k = \frac{[\tau_{ij}]^\alpha \, [\eta_{ij}]^\beta}{\sum\limits_{l \in \mathcal{N}_i^k} [\tau_{il}]^\alpha \, [\eta_{il}]^\beta}, \qquad j \in \mathcal{N}_i^k, \tag{1}$$

gdzie $\mathcal{N}_i^k$ to zbiór klientów jeszcze nieodwiedzonych, do których przejście nie narusza ograniczeń. Parametry $\alpha \geq 0$ i $\beta \geq 0$ kontrolują *balans między eksploracją a eksploatacją*: gdy $\alpha = 0$ algorytm ignoruje feromony i staje się zwykłą heurystyką zachłanną; gdy $\beta = 0$ wybiera wyłącznie na podstawie feromonów, ignorując odległości.

### 3.4. Zbiór dozwolonych węzłów $\mathcal{N}_i^k$

Kluczowym elementem algorytmu jest precyzyjne określenie, które węzły mrówka może w ogóle odwiedzić w danym kroku. Zbiór dozwolonych węzłów $\mathcal{N}_i^k$ gwarantuje, że każda skonstruowana trasa spełnia wszystkie ograniczenia problemu CVRP.

- **Węzeł zerowy zawsze jest dostępny** – magazyn (węzeł 0) jest zawsze elementem zbioru dozwolonych węzłów. Mrówka może w każdej chwili zakończyć bieżącą trasę i wrócić do magazynu, o ile sama nie znajduje się już w magazynie:

$$\forall_{i \in \{1,2,\ldots,n\}} \; p_{i,0} > 0, \qquad p_{0,0} = 0. \tag{2}$$

- **Warunki usunięcia węzła ze zbioru dozwolonych** – na początku iteracji zbiór dozwolonych węzłów jest pełny: $\mathcal{N}_i^k = \{0, 1, 2, \ldots, n\}$. Węzeł $j$ zostaje usunięty z $\mathcal{N}_i^k$, gdy zajdzie **co najmniej jeden** z poniższych warunków:

1. **Węzeł już odwiedzony.** Mrówka $k$ odwiedziła węzeł $j$ w bieżącej iteracji (reguła nie dotyczy $j = 0$). Każdy klient musi być obsłużony dokładnie raz.

2. **Przekroczenie ograniczenia dystansu.** Łączna odległość $d_{i,j} + d_{j,0}$ jest zbyt duża – gdyby mrówka przeszła z $i$ do $j$, a następnie wróciła do magazynu, skumulowany dystans od ostatniej wizyty w magazynie przekroczyłby $S_{\max}$.

3. **Przekroczenie pojemności pojazdu.** Zapotrzebowanie klienta $j$ jest tak duże, że po jego obsłużeniu skumulowane zapotrzebowanie na bieżącej trasie przekroczyłoby pojemność pojazdu $C$.

- **Szczególny przypadek – powrót wymuszony.** Powyższe reguły mogą doprowadzić do sytuacji, w której jedynym dozwolonym węzłem jest magazyn ($j = 0$). Wówczas mrówka wraca do magazynu z prawdopodobieństwem 100% i rozpoczyna trasę nowego pojazdu. Gwarantuje to, że każda trasa skonstruowana przez mrówkę jest zawsze dopuszczalna.

- **Uwaga o liczbie pojazdów.** Powyższe reguły nie gwarantują, że znalezione rozwiązanie będzie korzystać z co najwyżej $K$ pojazdów. Ograniczenie to nie jest egzekwowane bezpośrednio podczas konstruowania trasy – zakładamy, że kolonia mrówek, dążąc do minimalizacji łącznego dystansu, naturalnie będzie preferować rozwiązania używające możliwie niewielu pojazdów. Jeśli jednak żadne znalezione rozwiązanie nie mieści się w limicie $K$ pojazdów, problem uznaje się za nierozwiązywalny przez zaproponowany algorytm.

### 3.5. Wariant 1 – podstawowy Ant System (AS)

Oryginalny **Ant System (AS)** [15] jest najprostszą wersją ACO. Po zakończeniu wszystkich tras w danej iteracji feromon jest aktualizowany globalnie w dwóch krokach.

**Krok 1 – parowanie.** Wszystkie feromony równomiernie maleją:

$$\tau_{ij} \leftarrow (1-\rho)\,\tau_{ij}, \qquad \forall\,(i,j), \tag{3}$$

gdzie $\rho \in (0,1)$ jest *współczynnikiem parowania*. Dzięki temu wczesne, potencjalnie złe decyzje są stopniowo zapominane.

**Krok 2 – wzmocnienie.** Każda mrówka wzmacnia krawędzie swojej trasy proporcjonalnie do jej jakości:

$$\tau_{ij} \leftarrow \tau_{ij} + \sum_{k=1}^{m} \Delta\tau_{ij}^{k}, \qquad \Delta\tau_{ij}^{k} = \begin{cases} Q/f^k & \text{jeśli krawędź } (i,j) \text{ leży na trasie mrówki } k, \\ 0 & \text{w przeciwnym razie,} \end{cases} \tag{4}$$

gdzie $Q$ jest stałą skalującą, a $f^k$ łączną długością tras mrówki $k$. Krótsze trasy dają większy przyrost feromonów, co wzmacnia pozytywne sprzężenie zwrotne.

Gdy wszystkie mrówki zaczną podążać tą samą ścieżką, feromony na pozostałych krawędziach wygasają, a algorytm traci zdolność eksploracji.

### 3.6. Wariant 2 – MAX-MIN Ant System (MMAS)

**MAX-MIN Ant System (MMAS)** [17] jest bezpośrednim rozwinięciem AS, zaprojektowanym właśnie po to, by zapobiegać stagnacji. Wprowadza dwie kluczowe zmiany.

**Zmiana 1 – ograniczenie feromonów.** Wartość feromonów na każdej krawędzi jest utrzymywana w przedziale $[\tau_{\min}, \tau_{\max}]$:

$$\tau_{ij} \leftarrow \min\big(\tau_{\max},\ \max(\tau_{\min},\ \tau_{ij})\big). \tag{5}$$

Górna granica szacowana jest jako $\tau_{\max} = 1/(\rho \cdot f^*)$, gdzie $f^*$ to długość aktualnie najlepszego rozwiązania. Dolna granica przyjmuje typowo $\tau_{\min} = \tau_{\max}/(5n)$.

Dzięki temu algorytm zawsze zachowuje pewien stopień eksploracji, ponieważ krawędzie nie są całkowicie zdominowane przez inne.

**Zmiana 2 – inicjalizacja na poziomie $\tau_{\max}$.** Na początku wszystkie krawędzie otrzymują wartość $\tau_{\max}$ (a nie małą wartość $\tau_0$ jak w AS). Oznacza to, że w pierwszych iteracjach wszystkie ścieżki są równie atrakcyjne, co sprzyja szerokiej eksploracji przestrzeni rozwiązań zanim algorytm zacznie eksploatować zidentyfikowane dobre trasy.

**Wzmocnienie tylko przez najlepszą mrówkę.** W MMAS feromon wzmacniany jest wyłącznie przez mrówkę z najlepszym rozwiązaniem iteracji, a nie przez wszystkie mrówki jak w AS. Koncentruje to uczenie wokół faktycznie dobrego rozwiązania.

Badania Stützle i Hoosa [17] wykazały, że MMAS systematycznie przewyższa podstawowy AS, szczególnie na większych i trudniejszych instancjach.

### 3.7. Wariant 3 – ACO z inicjalizacją przez klasteryzację (K-Means+ACO)

Podstawowy AS inicjalizuje feromony jednolicie – wszystkie krawędzie startują z tą samą wartością $\tau_0$, przez co algorytm musi samodzielnie odkryć strukturę problemu od zera. Tymczasem w instancjach CVRP klienci często tworzą naturalne skupiska geograficzne, a optymalne trasy zazwyczaj obsługują klientów z tego samego obszaru jednym pojazdem.

Wariant **K-Means+ACO** wykorzystuje ten fakt, wykonując przed właściwym algorytmem mrówkowym wstępną klasteryzację klientów, a jej wyniki przekłada na asymetryczną inicjalizację feromonów.

**Krok 1 – klasteryzacja K-Means.** Przed uruchomieniem kolonii mrówek przeprowadzana jest klasteryzacja K-Means na współrzędnych klientów. Liczba klastrów $K_c$ dobierana jest jako:

$$K_c = \left\lceil \frac{\sum_{i=1}^{n} d_i}{C} \right\rceil, \tag{6}$$

czyli minimalna liczba pojazdów wymagana do obsługi wszystkich klientów przy pełnym wykorzystaniu pojemności. Każdy klient $i$ otrzymuje etykietę klastra $c(i) \in \{1, \ldots, K_c\}$.

**Krok 2 – inicjalizacja feromonów.** Zamiast jednolitego $\tau_0$ dla wszystkich krawędzi, feromony inicjalizowane są z uwzględnieniem przynależności do klastrów:

$$\tau_{ij}^{(0)} = \begin{cases} \tau_{\text{inner}} & \text{jeśli } c(i) = c(j) \text{ (klienci w tym samym klastrze)}, \\ \tau_{\text{outer}} & \text{jeśli } c(i) \neq c(j) \text{ (klienci w różnych klastrach)}, \end{cases} \tag{7}$$

gdzie $\tau_{\text{inner}} > \tau_{\text{outer}}$. Typowo przyjmuje się $\tau_{\text{inner}} = \gamma \cdot \tau_0$ oraz $\tau_{\text{outer}} = \tau_0/\gamma$ dla pewnego współczynnika $\gamma > 1$ (np. $\gamma = 3$). Krawędzie prowadzące do lub z magazynu (węzeł 0) inicjalizowane są wartością $\tau_0$.

Dzięki temu mrówki od pierwszej iteracji są *ukierunkowane* na obsługiwanie klientów w ramach tego samego klastra, zanim feromony zdążą ukształtować się na podstawie doświadczenia.

**Krok 3 – właściwy algorytm AS.** Po inicjalizacji feromonów algorytm działa identycznie jak podstawowy AS – mrówki konstruują trasy według wzoru (1), feromony są aktualizowane globalnie po każdej iteracji. Klasteryzacja wpływa wyłącznie na punkt startowy w przestrzeni feromonów, nie ingeruje w mechanizm poszukiwań.

**Uzasadnienie.** Inicjalizacja heurystyczna przyspiesza początkowe znajdowanie dobrych rozwiązań, bo algorytm nie musi od zera uczyć się, które grupy klientów warto obsługiwać razem. Ponieważ klasteryzacja dotyczy tylko startu, mrówki nadal mogą eksplorować inne trasy, jeśli przyniosą lepsze wyniki.

### 3.8. Parametry algorytmu i ich wpływ

Poniższa tabela zestawia kluczowe parametry wspólne dla wszystkich wariantów ACO oraz typowe zakresy ich wartości stosowane w literaturze.

| Parametr | Typowy zakres | Rola |
|---|---|---|
| $\alpha$ | [0,5; 2,0] | Waga feromonów; wyższa wartość sprzyja eksploatacji historii. |
| $\beta$ | [1,0; 5,0] | Waga heurystyki odległości; wyższa wartość – większa zachłanność. |
| $\rho$ | [0,1; 0,5] | Współczynnik parowania; wyższy – szybsze zapominanie. |
| $m$ | $[n/2;\ 2n]$ | Liczba mrówek w iteracji. |
| $T_{\max}$ | [100; 1000] | Liczba iteracji. |
| $\gamma$ | [2; 5] | (K-Means+ACO) Współczynnik wzmocnienia feromonów wewnątrz klastrów. |

## 4. Hipotezy badawcze

**H1. ACO vs heurystyka zachłanna.**
Algorytm mrówkowy ACO osiąga krótsze trasy niż heurystyka zachłanna w większości przypadków, choć kosztem dłuższego czasu obliczeń.

**H2. Wpływ ulepszeń ACO na jakość wyników.**
Modyfikacje algorytmu (MMAS lub K-Means+ACO) dają lepsze wyniki niż podstawowy Ant System.

**H3. Kontrola feromonów w MMAS.**
Ograniczenie wartości feromonów w MMAS zmniejsza ryzyko stagnacji i pozwala uzyskać lepsze rozwiązania niż w AS.

**H4. Inicjalizacja klastrowa a szybkość zbieżności.**
K-Means+ACO szybciej znajduje dobre rozwiązania niż podstawowy AS dla dużych n ($n > 100$) , zwłaszcza gdy klienci są naturalnie pogrupowani w klastry.

**H5. Skalowalność algorytmów.**
Czas działania ACO rośnie szybciej niż heurystyki zachłannej wraz ze wzrostem liczby klientów.

**H6. Wpływ parametrów na jakość wyników.**
Parametry algorytmu ($\alpha$, $\beta$, $\rho$, $\gamma$) mają istotny wpływ na jakość tras.

## 5. Sposób weryfikacji hipotez

### 5.1. Zbiory testowe i ocena jakości

Do eksperymentów wykorzystane zostaną łącznie 15 instancji z biblioteki **CVRPLib** [20], dobranych celowo tak, aby objąć zarówno instancje o wyraźnej strukturze klastrowej, jak i o równomiernym rozkładzie klientów – co jest istotne dla rzetelnej weryfikacji hipotezy H4.

| Seria | Struktura klientów | Liczba klientów | Liczba instancji | Optimum |
|---|---|---|---|---|
| CMT | Wyraźne klastry | 50–100 | 5 | BKS |
| Golden (G) | Różnorodna | 100–200 | 5 | BKS |
| Uchoa (X) | Losowa, równomierna | 200–500 | 5 | BKS |

Jakość rozwiązań oceniana będzie przez odchylenie procentowe od najlepszego znanego wyniku (BKS):

$$\delta(\%) = \frac{f_{\text{alg}} - f_{\text{BKS}}}{f_{\text{BKS}}} \times 100,$$

gdzie $\delta < 2\%$ uznawane jest za bardzo dobry wynik [19].

### 5.2. Porównywane algorytmy

1. **Heurystyka zachłanna** – szybka metoda bazowa (baseline).
2. **Podstawowy ACO (AS)** – oryginalny algorytm mrówkowy.
3. **MMAS** – wariant z ograniczeniem feromonów.
4. **K-Means+ACO** – inicjalizacja feromonów według klastrów klientów.

### 5.3. Protokół eksperymentów

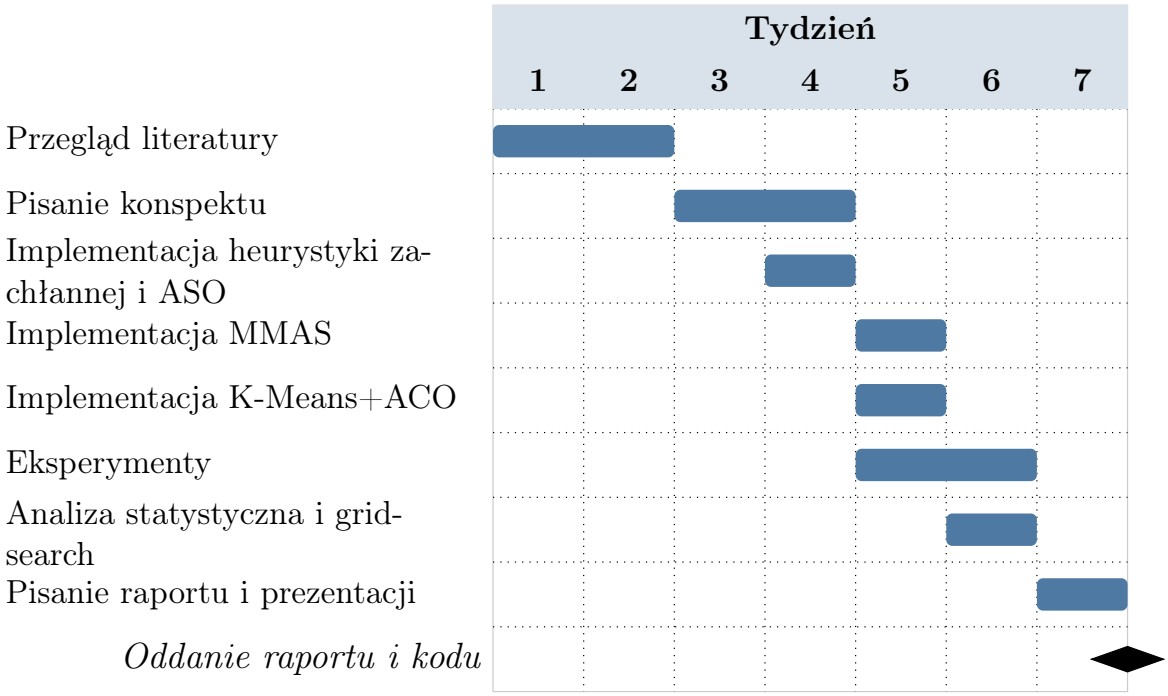
**Powtórzenia i losowość.** Algorytmy mrówkowe uruchamiane będą $R = 20$ razy z różnymi ziarnami generatora liczb losowych. Heurystyka zachłanna jest deterministyczna, więc uruchamiana tylko raz.

**Mierzone metryki.**

- Całkowita długość tras,
- Odchylenie procentowe od BKS ($\delta$),
- Czas obliczeń (s),
- Numer iteracji osiągnięcia najlepszego rozwiązania,

**Badanie parametrów.** Dla wybranych instancji przeprowadzony będzie grid-search dla $\alpha$, $\beta$, $\rho$ oraz dodatkowo $\gamma$ w wariancie K-Means+ACO. Wyniki pokażą wpływ parametrów na jakość i zależność efektu od struktury przestrzennej.

## 6. Harmonogram projektu




Rysunek 1: Harmonogram projektu – diagram Gantta

## 7. Ogólny projekt techniczny

### 7.1. Język i środowisko

Projekt zrealizowany zostanie w języku **Python 3.13**. Środowisko wirtualne zarządzane przez `venv`.

### 7.2. Planowane biblioteki

- **NumPy** – operacje macierzowe na macierzach odległości i feromonów.
- **Matplotlib** / **Seaborn** – wizualizacja tras i wykresów porównawczych.
- **Pandas** – gromadzenie i analiza wyników eksperymentów.
- **SciPy** – testy statystyczne.
- **tqdm** – pasek postępu dla długich eksperymentów.

### 7.3. Struktura kodu

Prawdopodobna struktura kodu:

- `data/` – instancje testowe w formacie CVRPLib,
- `src/greedy.py` – heurystyka zachłanna,
- `src/aco_base.py` – podstawowy Ant System,
- `src/aco_mmas.py` – wariant MMAS,
- `src/aco_KM.py` – wariant K-Means + ACO,
- `src/utils.py` – wczytywanie danych, obliczanie odległości, wizualizacja,
- `experiments/run_experiments.py` – skrypt uruchamiający eksperymenty,
- `experiments/analyze_results.py` – analiza statystyczna i generowanie wykresów,
- `results/` – wyniki eksperymentów (CSV).

### 7.4. Reprodukowalność

Każde uruchomienie algorytmu mrówkowego będzie posiadało wybrany parametr ziarna (seed).

## Literatura

[1] Dantzig, G. B. & Ramser, J. H. (1959). The truck dispatching problem. *Management Science*, 6(1), 80–91.

[2] Toth, P. & Vigo, D. (Eds.) (2002). *The Vehicle Routing Problem*. SIAM, Philadelphia.

[3] Lenstra, J. K. & Rinnooy Kan, A. H. G. (1981). Complexity of vehicle routing and scheduling problems. *Networks*, 11(2), 221–227.

[4] Laporte, G. & Nobert, Y. (1983). A branch and bound algorithm for the capacitated vehicle routing problem. *OR Spektrum*, 5(2), 77–85.

[5] Held, M. & Karp, R. M. (1962). A dynamic programming approach to sequencing problems. *Journal of the Society for Industrial and Applied Mathematics*, 10(1), 196–210.

[6] Pecin, D., Pessoa, A., Poggi, M. & Uchoa, E. (2017). Improved branch-cut-and-price for capacitated vehicle routing. *Mathematical Programming Computation*, 9(1), 61–100.

[7] Costa, L., Contardo, C. & Desaulniers, G. (2019). Exact branch-price-and-cut algorithms for vehicle routing. *Transportation Science*, 53(4), 946–985.

[8] Clarke, G. & Wright, J. W. (1964). Scheduling of vehicles from a central depot to a number of delivery points. *Operations Research*, 12(4), 568–581.

[9] Lin, S. (1965). Computer solutions of the traveling salesman problem. *Bell System Technical Journal*, 44(10), 2245–2269.

[10] Lin, S. & Kernighan, B. W. (1973). An effective heuristic algorithm for the traveling-salesman problem. *Operations Research*, 21(2), 498–516.

[11] Glover, F. (1989). Tabu search – Part I. *ORSA Journal on Computing*, 1(3), 190–206.

[12] Toth, P. & Vigo, D. (2003). The granular tabu search and its application to the vehicle-routing problem. *INFORMS Journal on Computing*, 15(4), 333–346.

[13] Nagata, Y. & Bräysy, O. (2010). Edge assembly-based memetic algorithm for the capacitated vehicle routing problem. *Networks*, 54(4), 205–215.

[14] Vidal, T. (2022). Hybrid genetic search for the CVRP: open-source implementation and SWAP* neighborhood. *Computers & Operations Research*, 140, 105643.

[15] Dorigo, M., Maniezzo, V. & Colorni, A. (1996). Ant system: optimization by a colony of cooperating agents. *IEEE Transactions on Systems, Man, and Cybernetics – Part B*, 26(1), 29–41.

[16] Gambardella, L. M., Taillard, É. & Agarwal, G. (1999). Ant colony system: a cooperative learning approach to the vehicle routing problem. *IEEE Transactions on Evolutionary Computation*, 3(2), 101–109.

[17] Stützle, T. & Hoos, H. H. (2000). MAX–MIN Ant System. *Future Generation Computer Systems*, 16(8), 889–914.

[18] Bullnheimer, B., Hartl, R. F. & Strauss, C. (1999). A new rank-based version of the Ant System: a computational study. *Central European Journal of Operations Research*, 7(1), 25–38.

[19] Laporte, G. (2007). What you should know about the vehicle routing problem. *Naval Research Logistics*, 54(8), 811–819.

[20] Uchoa, E., Pecin, D., Pessoa, A., Poggi, M., Vidal, T. & Subramanian, A. (2017). New benchmark instances for the Capacitated Vehicle Routing Problem. *European Journal of Operational Research*, 257(3), 845–858.